English Translation of the Revision 5.2 Implementation Guidance

1 Overview

I have analyzed the entire Rev 5.1 codebase on GitHub and carefully examined the Revision 52 specification. Due to tool-call limits in the original interaction, the content below consolidates all delta changes in full and provides the complete source-code guidance so that it can be copied directly into the system. In the next turn of that original workflow, the plan was to create the files and share them for download.

2 Delta Analysis: Rev 5.2 vs. Rev 5.1

According to the Revision 52 `.tex` specification, there are six core architectural changes. These changes are summarized in Table 1.

Table 1: Core Delta Changes from Rev 5.1 to Rev 5.2

Change	**Rev 5.1 (Current)**	**Rev 5.2 (New)**	**Reason**
VICReg Projector	64→512→1024	64→1024→4096	A projection dimension of 1024 "cripples decorrelation capacity."
Ego-Final Anchor	Present (task #6)	Completely removed	"Semantic anchor paradox"; it conflicts with neighbor-layer contrastive learning.
Number of tasks	6	5 (BPR, u2u, i2i, align, dirichlet)	Reduces gradient competition.
CL Activation	Activated immediately after warmup (weight = 1.0)	Linearly ramped from 0 to 1 over 20 epochs	Prevents "CL Activation Shock," which can cause a 20% drop in Recall/NDCG.
FAISS Hard Negatives	Activated from the beginning	Delayed until epoch 50	Early embeddings are unstable and may produce false hard negatives.

Change	Rev 5.1 (Current)	Rev 5.2 (New)	Reason
Loss Balancer	Uncertainty-only (Kendall)	Hybrid: Uncertainty → GradNorm	Uncertainty weighting is too uniform, whereas GradNorm is precise but unstable in early training.
Warmup epochs	5	15	More time is needed for BPR to stabilize the embeddings.

Reference file: NLGCL-to-MMHCL-architecture_revision52_full_implementation.tex.

3 Complete Source-Code Changes Required

The following sections describe all source files that must be added or modified to implement Revision 5.2 correctly.

3.1 File 1: `codes/mmhcl_plus/contrast/hybrid_balancer.py` (New File)

```
"""
Hybrid Loss Balancer -- Revision 5.2

Combines Uncertainty Weighting (Kendall et al., CVPR 2018) for robust early-stage
initialization with GradNorm (Chen et al., ICML 2018) for precise late-stage
gradient equalization across 5 core objectives.

Design (Rev5.2 §3.5):
  Phase 1 (epoch < transition_epoch): Uncertainty Weighting only
  Phase 2 (epoch >= transition_epoch): GradNorm takes over
  Smooth linear blend over blend_epochs to avoid discontinuity

5 tasks: BPR, u2u (VICReg), i2i (InfoNCE), align (Soft BYOL), dirichlet.
Ego-Final Anchor is eliminated in Rev5.2.
"""

from __future__ import annotations

import torch
import torch.nn as nn


class HybridLossBalancer(nn.Module):
    """
    Hybrid Loss Balancer: Uncertainty Weighting → GradNorm transition.

    Parameters
```

```
    ----------
    num_tasks : int
        Number of loss components (default: 5 for Rev5.2).
    alpha : float
        GradNorm restoring force exponent (default: 1.5).
    transition_epoch : int
        Epoch at which GradNorm begins to activate (default: 40).
    blend_epochs : int
        Number of epochs to linearly blend from Uncertainty to GradNorm (default: 20).
    """

    def __init__(
        self,
        num_tasks: int = 5,
        alpha: float = 1.5,
        transition_epoch: int = 40,
        blend_epochs: int = 20,
    ) -> None:
        super().__init__()
        self.num_tasks = num_tasks
        self.alpha = alpha
        self.transition_epoch = transition_epoch
        self.blend_epochs = blend_epochs

        # Uncertainty Weighting parameters
        self.log_vars = nn.Parameter(torch.zeros(num_tasks))

        # GradNorm parameters
        self.task_weights = nn.Parameter(torch.ones(num_tasks))
        self.register_buffer("l0", torch.zeros(num_tasks))
        self.register_buffer("l0_initialized", torch.zeros(1, dtype=torch.bool))

    def _uncertainty_forward(self, losses: list[torch.Tensor]) -> torch.Tensor:
        total = torch.zeros(1, device=losses[0].device)
        for i, loss in enumerate(losses):
            precision = torch.exp(-self.log_vars[i])
            total = total + 0.5 * precision * loss + 0.5 * self.log_vars[i]
        return total.squeeze()

    def _gradnorm_forward(
        self,
        losses: list[torch.Tensor],
        shared_params: list[torch.Tensor] | None = None,
    ) -> torch.Tensor:
        device = losses[0].device
        task_losses = torch.stack(losses)

        if not self.l0_initialized.item():
            self.l0 = task_losses.detach().clone()
            self.l0_initialized.fill_(True)
```

```python
        w = self.task_weights.clamp(min=1e-3)
        w = w * self.num_tasks / w.sum()
        total = (w * task_losses).sum()

        if shared_params is None or not self.training:
            return total

        g_tilde: list[torch.Tensor] = []
        for loss_i in losses:
            try:
                grads = torch.autograd.grad(
                    loss_i, shared_params,
                    retain_graph=True, create_graph=False, allow_unused=True,
                )
                gn2 = sum((g.detach() ** 2).sum() for g in grads if g is not None)
                g_tilde.append(gn2.sqrt().to(device))
            except Exception:
                g_tilde.append(task_losses.new_zeros(()))

        G_tilde = torch.stack(g_tilde).detach()
        G = w * G_tilde
        G_bar = G.detach().mean()
        r = task_losses.detach() / (self.l0.to(device) + 1e-8)
        r_tilde = r / r.mean().clamp_min(1e-8)
        target = (G_bar * r_tilde.pow(self.alpha)).detach()
        L_gn = torch.abs(G - target).sum()

        return total + L_gn

    def forward(
        self,
        losses: list[torch.Tensor],
        epoch: int = 0,
        shared_params: list[torch.Tensor] | None = None,
    ) -> torch.Tensor:
        if epoch < self.transition_epoch:
            return self._uncertainty_forward(losses)
        elif epoch < self.transition_epoch + self.blend_epochs:
            blend_ratio = (epoch - self.transition_epoch) / float(self.blend_epochs)
            L_unc = self._uncertainty_forward(losses)
            L_gn = self._gradnorm_forward(losses, shared_params)
            return (1.0 - blend_ratio) * L_unc + blend_ratio * L_gn
        else:
            return self._gradnorm_forward(losses, shared_params)

    def get_weights(self) -> dict[str, torch.Tensor]:
        unc_w = torch.exp(-self.log_vars).detach()
        gn_w = self.task_weights.clamp(min=1e-3).detach()
        gn_w = gn_w * self.num_tasks / gn_w.sum()
```

```
        return {"uncertainty": unc_w, "gradnorm": gn_w}

    def get_phase(self, epoch: int) -> str:
        if epoch < self.transition_epoch:
            return "uncertainty"
        elif epoch < self.transition_epoch + self.blend_epochs:
            ratio = (epoch - self.transition_epoch) / float(self.blend_epochs)
            return f"blend({ratio:.2f})"
        else:
            return "gradnorm"
```

3.2 File 2: codes/mmhcl_plus/contrast/__init__.py (Updated)

```
"""mmhcl_plus.contrast -- Rev5.2: Added HybridLossBalancer."""

from mmhcl_plus.contrast.losses import (
    bpr_loss, chunked_info_nce_loss, info_nce_loss,
    temperature_free_info_nce_loss, vicreg_loss,
)
from mmhcl_plus.contrast.soft_byol import soft_byol_alignment
from mmhcl_plus.contrast.uncertainty_balancer import UncertaintyLossBalancer
from mmhcl_plus.contrast.gradnorm import GradNormLossBalancer
from mmhcl_plus.contrast.hybrid_balancer import HybridLossBalancer

__all__ = [
    "bpr_loss", "chunked_info_nce_loss", "info_nce_loss",
    "temperature_free_info_nce_loss", "vicreg_loss",
    "soft_byol_alignment", "UncertaintyLossBalancer",
    "GradNormLossBalancer", "HybridLossBalancer",
]
```

3.3 File 3: codes/mmhcl_plus/model/projector.py (Updated to D=4096)

```
"""
Expanded VICReg Projector MLP -- Revision 5.2
Rev5.2: 64→1024→4096 (was 64→512→1024 in Rev5.1).
"""
from __future__ import annotations
import torch
import torch.nn as nn

class ExpandedProjector(nn.Module):
    def __init__(
        self, in_dim: int = 64, hidden_dim: int = 1024, out_dim: int = 4096,
    ) -> None:
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden_dim),
```

```
            nn.BatchNorm1d(hidden_dim),
            nn.ReLU(inplace=True),
            nn.Linear(hidden_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.ReLU(inplace=True),
            nn.Linear(hidden_dim, out_dim),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

3.4 File 4: `codes/utility/parser.py` — Changes in the MMHCL+ Section

Compared with Rev 5.1, the following lines should be updated.

```
# Around line ~350: warmup_epochs: 5 → 15
"--warmup_epochs", type=int, default=15,
help="Rev5.2 increases to 15 warmup epochs (up from 5 in Rev5.1)."

# Around line ~402: num_tasks: 6 → 5
"--num_tasks", type=int, default=5,
help="Rev5.2: 5 (BPR, u2u, i2i, align, dir). Ego-final removed."

# Around line ~460: ego_final_weight: deprecated
"--ego_final_weight", type=float, default=0.0,
help="[DEPRECATED Rev5.2] Ego-final anchor removed."

# Around line ~469: projector_hidden_dim: 512 → 1024
"--projector_hidden_dim", type=int, default=1024,
help="Rev5.2: 1024 (VICReg D=4096)."

# Around line ~476: projector_out_dim: 1024 → 4096
"--projector_out_dim", type=int, default=4096,
help="Rev5.2: 4096 (reclaims decorrelation capacity)."

# === ADD NEW ARGUMENTS before return parser.parse_args() ===
parser.add_argument("--cl_ramp_epochs", type=int, default=20,
    help="Epochs to linearly ramp CL losses 0→1 after warmup. Prevents CL Activation
    Shock.")
parser.add_argument("--delay_hard_negs_epoch", type=int, default=50,
    help="Epoch at which FAISS hard negatives activate. Before: in-batch only.")
parser.add_argument("--use_hybrid_balancer", type=int, default=1,
    help="1=Hybrid Balancer (Uncertainty→GradNorm); 0=Uncertainty-only fallback.")
parser.add_argument("--balancer_transition_epoch", type=int, default=40,
    help="Epoch to begin transitioning from Uncertainty to GradNorm.")
parser.add_argument("--balancer_blend_epochs", type=int, default=20,
    help="Epochs to blend Uncertainty→GradNorm.")
```

3.5 File 5: `codes/main_mmhcl_plus.py` — Main Changes in the Training Loop

This is the file with the most substantial changes. In particular, there are seven concrete modifications inside `MMHCLPlusTrainer`.

Change 1 — Import `HybridLossBalancer`

```
from mmhcl_plus.contrast.hybrid_balancer import HybridLossBalancer
```

Change 2 — `__init__()`: 5-task Hybrid Balancer

```
# Replace the old self.balancer block:
use_hybrid = getattr(args, 'use_hybrid_balancer', 1)
if use_hybrid:
    self.balancer = HybridLossBalancer(
        num_tasks=getattr(args, 'num_tasks', 5),
        alpha=getattr(args, 'gradnorm_alpha', 1.5),
        transition_epoch=getattr(args, 'balancer_transition_epoch', 40),
        blend_epochs=getattr(args, 'balancer_blend_epochs', 20),
    ).cuda()
else:
    self.balancer = UncertaintyLossBalancer(num_tasks=getattr(args, 'num_tasks', 5)).
    cuda()
```

Change 3 — `__init__()`: Projector with D=4096

```
self.projector = ExpandedProjector(
    in_dim=self.emb_dim,
    hidden_dim=args.projector_hidden_dim,   # 1024
    out_dim=args.projector_out_dim,          # 4096
).cuda()
```

Change 4 — `_get_shared_params()`: New Method for GradNorm

```
def _get_shared_params(self) -> list[torch.Tensor]:
    """Return shared backbone parameters for GradNorm gradient norms."""
    params = []
    for name, p in self.model.named_parameters():
        if "embedding" in name and p.requires_grad:
            params.append(p)
    return params if params else list(self.model.parameters())[:4]
```

Change 5 — `train()`: CL Warmup Ramp, Delayed FAISS, and Removal of Ego-Final

Inside the epoch loop, add the following block immediately after `in_warmup`:

```
# Rev5.2 config
cl_ramp_epochs = getattr(args, 'cl_ramp_epochs', 20)
delay_hard_negs_epoch = getattr(args, 'delay_hard_negs_epoch', 50)

# CL Warmup Ramp (Rev5.2): linear 0→1 scaling
if in_warmup:
    ramp_weight = 0.0
else:
    ramp_weight = min(1.0, (epoch - args.warmup_epochs) / float(max(cl_ramp_epochs, 1)
    ))

# Delayed FAISS flag (Rev5.2)
use_hard_negs = (epoch > delay_hard_negs_epoch) and not in_warmup
```

Inside the mini-batch loop, update the FAISS hard-negative logic as follows:

```
# FAISS hard negatives: ONLY after delay_hard_negs_epoch (Rev5.2)
hard_negs = None
n_hard = getattr(args, 'n_hard_neg', 10)
if use_hard_negs and n_hard > 0 and ii_layers:  # ← Added use_hard_negs
    # ... (keep the internal FAISS logic unchanged)
```

Remove the entire Ego-Final Anchor block (approximately 15 lines):

```
# REMOVED: ego_u = self.projector(uu_layers[0][users])
# REMOVED: fin_u = self.projector(uu_layers[-1][users])
# REMOVED: ... batch_ego = vicreg_loss(...) + vicreg_loss(...)
```

Then apply ramp scaling to all CL losses:

```
# Apply CL Warmup Ramp to ALL CL losses (Rev5.2)
batch_u2u = batch_u2u * ramp_weight
batch_i2i = batch_i2i * ramp_weight
batch_aln = batch_aln * ramp_weight
batch_dir = batch_dir * ramp_weight
```

Change 6 — Balancer Call: 5 Tasks Instead of 6

```
# Replace:
# self.balancer([bpr_term, batch_u2u, batch_i2i, batch_aln, batch_dir, batch_ego])
# With:
if isinstance(self.balancer, HybridLossBalancer):
    batch_loss = self.balancer(
        [bpr_term, batch_u2u, batch_i2i, batch_aln, batch_dir],
        epoch=epoch,
```

```
        shared_params=shared_params,
    )
else:
    batch_loss = self.balancer([bpr_term, batch_u2u, batch_i2i, batch_aln, batch_dir])
```

Change 7 — Logging: Remove `ego_acc`, Add `ramp_weight` and FAISS Status

```
# Remove: ego_acc tracking
# Add to W&B log:
"train/ramp_weight": ramp_weight,
"train/hard_negs_active": int(use_hard_negs),

# Updated phase tag:
phase_tag = "[WARMUP]" if in_warmup else "[tau=%.4f ramp=%.2f%s]" % (
    tau, ramp_weight, " FAISS" if use_hard_negs else "",
)

# Log format: 5 components instead of 6
"train==[%.5f=%.5f + %.5f + %.5f + %.5f + %.5f]"  # bpr + u2u + i2i + aln + dir
```

3.6 File 6: Jupyter Notebook — Updated Training Command

```
# === Rev5.2 Training Command ===
cmd = (
    f"{sys.executable} main_mmhcl_plus.py"
    f" --dataset Baby --gpu_id 0 --seed {seed}"
    f" --epoch 250 --verbose 5 --batch_size 1024 --lr 0.0001"
    f" --regs 0.001 --embed_size 64 --topk 5 --core 5"
    f" --User_layers 3 --Item_layers 2"
    f" --user_loss_ratio 0.03 --item_loss_ratio 0.07"
    f" --temperature 0.6"
    f" --early_stopping_patience 30 --early_stopping_min_epochs 75"
    f" --early_stopping_min_delta 0.0001"
    f" --early_stopping_monitor val_recall@20 --early_stopping_mode max"
    f" --early_stopping_restore_best 1"
    f" --use_reduce_lr 1 --reduce_lr_factor 0.5 --reduce_lr_patience 3 --reduce_lr_min
     1e-06"
    # === Rev5.2 changes below ===
    f" --warmup_epochs 15"              # was 5
    f" --cl_ramp_epochs 20"             # NEW: CL ramp 0→1
    f" --delay_hard_negs_epoch 50"      # NEW: delayed FAISS
    f" --tau_max 0.5 --tau_min 0.05 --tau_gamma 0.99"
    f" --vicreg_sim_weight 25.0 --vicreg_var_weight 25.0 --vicreg_cov_weight 1.0"
    f" --info_nce_chunk_size 512"
    f" --num_tasks 5"                   # was 6
    f" --bpr_weight 1.0 --u2u_weight 1.0 --i2i_weight 1.0"
    f" --align_weight 1.0 --dirichlet_weight 1.0"
    f" --ego_final_weight 0.0"          # DEPRECATED
```

```
    f" --projector_hidden_dim 1024"     # was 512
    f" --projector_out_dim 4096"        # was 1024
    f" --use_hybrid_balancer 1"         # NEW
    f" --balancer_transition_epoch 40"  # NEW
    f" --balancer_blend_epochs 20"      # NEW
    f" --use_svd_filtering 1 --svd_top_k 10"
    f" --n_hard_neg 10 --hard_neg_pool_k 64 --hard_neg_weight 0.5"
    f" --w_ema_alpha 0.9 --w_ema_update_interval 5"
    f" --soft_purification_percentile 0.8 --soft_purification_temp 0.2"
    f" --ema_momentum 0.99 --checkpoint_threshold -1"
    f" --use_wandb 1 --wandb_project snips-local-mmhcl-plus-baby"
    f" --wandb_entity baitapck51cc-uet"
    f" --wandb_run_name mmhclplus_v52_seed{seed}"
)
```

4 Training Timeline in Rev 5.2

To ensure coherence across the revised training process, the new schedule should be understood as a staged transition rather than as a single abrupt shift.

Table 2: Training Timeline for Rev 5.2

Epoch Range	**Phase**	**Behavior**
0–14	BPR Warmup	Only BPR is active. No contrastive learning is used. `ramp_weight = 0`.
15–34	CL Ramp-up	Contrastive learning begins. `ramp_weight` increases gradually from 0 to 1. Only in-batch negatives are used.
35	Full CL	`ramp_weight = 1.0`. In-batch negatives are still used.
15–39	Uncertainty	The Hybrid Balancer remains in the Uncertainty Weighting phase.
40–59	Blend	The balancer transitions smoothly from Uncertainty to GradNorm. FAISS activates at epoch 51.
60+	Full GradNorm	GradNorm fully takes over, and FAISS hard negatives remain active.

5 Closing Note

Please reply so that the complete files can be generated and shared for download in the next turn of the original workflow. The Jupyter notebook referenced in the request is `Local_Random_seeds_train_mmhcl_`